You can query a global secondary index like you would query the table. You can imagine a global secondary index as a read-only DynamoDB table that is automatically maintained by DynamoDB: whenever you change the parent table, all indexes are asynchronously (eventually consistent!) updated as well.

In our example, we will create a global secondary index for the table `todo-tasks` which uses the `category` as the partition key and the `tid` as the sort key. Doing so allows us to query tasks by category. Figure 12.4 shows how a global secondary index works.

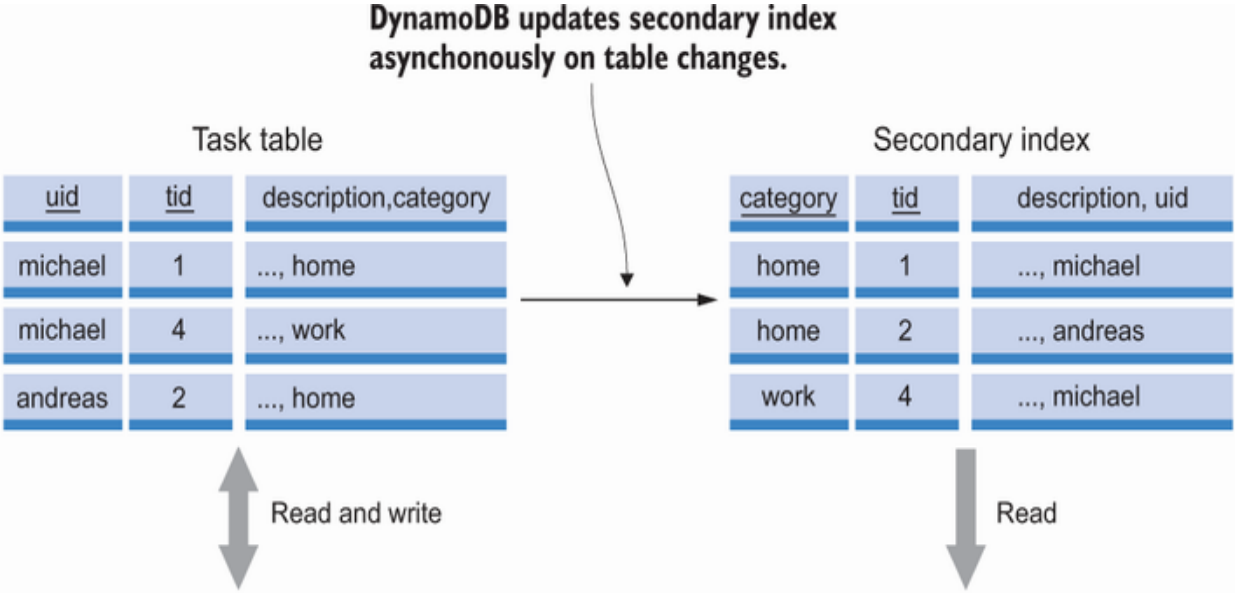


Figure 12.4 A global secondary index contains a copy (projection) of your table's data to provide fast lookup on another key.

A global secondary index comes at a price: the index requires storage (the same cost as for the original table). You must provision additional write-capacity units for the index as well, because a write to your table will cause a write to the global secondary index as well.